Object Oriented Design and Programming

Dr. Sanjeev Patel

Asst. Professor Grade-I

Department of Computer Science and Engineering

National Institute of Technology Rourkela, Odisha

Outline

- Java Collections
- Lists
- Stream Classes
- Input/Output Stream
- Reader/Writer
- Thread Class
- Creating a Thread
- Synchronization

Java Collections

- Collection Enables you to work with groups of objects;
  - it is at the top of the collections hierarchy
- The **Collection** interface is the foundation upon which the collections framework is built.
  - It declares the core methods that all collections will have.
- boolean add(Object *obj*) -Adds *obj* to the invoking collection.
  - Returns **true** if *obj* was added to the collection.
  - Returns **false** if *obj* is already a member of the collection, or if the collection does not allow duplicates.
- boolean contains(Object *obj*) Returns **true** if *obj* is an element of the invoking collection. Otherwise, returns **false**.
- int hashCode( ) Returns the hash code for the invoking collection.

❑ The List Interface:-

- List extends the Collection interface.
- It represents a sequence of elements.
- Elements are accessed using a zero-based index.
- A list can contain duplicate elements.
- Elements can be inserted or accessed by their position in the list.

Method	Description
add(int index, Object obj)	Inserts an element at the specified index and shifts existing elements.
addAll(int index, Collection c)	Inserts all elements of collection **c** at the specified index.
get(int index)	Returns the element at the specified index.
listIterator()	Returns an iterator starting from the beginning of the list
listIterator(int index)	Returns an iterator starting from the specified index.
remove(int index)	Removes the element at the specified index and shifts remaining elements.
subList(int start, int end)	Returns a portion of the list from **start** to **end-1**.
indexOf(Object obj)	Returns the index of the first occurrence of the object; returns **-1** if not found.

❑ The ArrayList Class

- **ArrayList** extends **AbstractList** and implements the **List interface**.
- It supports **dynamic arrays** that can grow or shrink automatically.
- Unlike normal arrays, ArrayList size is not fixed.
- ArrayList stores object references.
- It is created with an initial capacity, but it can increase automatically when needed.
- When elements are removed, the size of the ArrayList **can decrease**.
- ArrayList has the constructors shown here:

ArrayList( )

ArrayList(Collection c)

ArrayList(int capacity)

```
// Demonstrate ArrayList.
import java.util.*;
class ArrayListDemo {
public static void main(String args[]) {
// create an array list
ArrayList al = new ArrayList();
System.out.println("Initial size of al: " +
al.size());
// add elements to the array list
al.add("C");
al.add("A");
al.add("E");
al.add("B");
al.add("D");
al.add("F");
al.add(1, "A2");
System.out.println("Size of al after
additions: " +
al.size());
// display the array list
System.out.println("Contents of al: " +
al);
// Remove elements from the array list
al.remove("F");
al.remove(2);
System.out.println("Size of al after
deletions: " +
al.size());
System.out.println("Contents of al: " +
al);
}
}
```

❑ The LinkedList Class

- LinkedList extends **AbstractSequentialList** and implements the List interface.
- It provides a linked-list data structure.
- Constructors:

LinkedList() :- creates an empty linked list.

LinkedList(Collection c) :- creates a linked list with elements from collection c**.**

- Important methods:

addFirst(Object obj):- adds an element at the beginning of the list.

addLast(Object obj):- adds an element at the end of the list.

getFirst():- returns the first element.

getLast():- returns the last element.

removeFirst():- removes the first element.

removeLast():- removes the last element.

```
// Demonstrate LinkedList.
import java.util.*;
class LinkedListDemo {
public static void main(String args[]) {
// create a linked list
LinkedList ll = new LinkedList();
// add elements to the linked list
ll.add("F");
ll.add("B");
ll.add("D");
ll.add("E");
ll.add("C");
ll.addLast("Z");
ll.addFirst("A");
ll.add(1, "A2");
System.out.println("Original contents of ll: " + ll);
// remove elements from the linked list
ll.remove("F");
ll.remove(2);
System.out.println("Contents of ll after deletion: "
+ ll);
// remove first and last elements
ll.removeFirst();
ll.removeLast();
System.out.println("ll after deleting first and last:
"
+ ll);
// get and set a value
Object val = ll.get(2);
ll.set(2, (String) val + " Changed");
System.out.println("ll after change: " + ll);
}
}
```

❑ Stream classes:-

- Java’s stream-based I/O is based on four abstract classes:
  - InputStream
  - OutputStream
  - Reader
  - Writer
- These classes define the basic functionality for all stream classes.
- Character streams are used when working with characters or strings.
- Byte streams are used for byte-oriented input and output operations.
- Byte Stream handle any type of data, including binary data.
- The byte stream hierarchy is based on two main classes:
  - InputStream
  - OutputStream

❑ InputStream

- InputStream is an abstract class used for byte input.
- It defines the basic methods for reading byte data.
- All methods in InputStream throw IOException when an error occurs.

Method	Description
int available()	Returns the number of bytes available for reading
void close()	Closes the input stream; further reading causes an IOException.
void mark(int numBytes)	Marks the current position in the input stream
boolean markSupported()	Checks if mark() and reset() are supported.
int read()	Reads the next byte; returns **-1** at end of file.
int read(byte buffer[])	Reads bytes into buffer and returns number of bytes read.
int read(byte buffer[], int offset, int numBytes)	Reads bytes into buffer starting from offset.
void reset()	Returns to the previously marked position.
long skip(long numBytes)	Skips the specified number of bytes in the input stream.

❑ OutputStream

- It is used to write byte data to an output destination.
- It is part of the byte stream hierarchy.
- All methods in OutputStream return void.
- If an error occurs, the methods throw an IOException.

Method	Description
void close()	Closes the output stream; further write attempts cause an IOException.
void flush()	Clears output buffers and finalizes the output.
void write(int b)	Writes a single byte to the output stream.
void write(byte buffer[])	Writes an entire byte array to the output stream.
void write(byte buffer[], int offset, int numBytes)	Writes a specific number of bytes from the array starting at the given offset.

❑ FileInputStream

- FileInputStream is used to read bytes from a file.
- It creates an InputStream connected to a file.
- It is commonly used for byte-oriented file input operations.

Constructors :- **FileInputStream(String filepath)** **FileInputStream(File fileObj)**

```
import java.io.*;
class FileInputStreamDemo {
public static void main(String args[]) throws Exception {
int size;
InputStream f =
new FileInputStream("FileInputStreamDemo.java");
System.out.println("Total Available Bytes: " +
(size = f.available()));
int n = size/40;
System.out.println("First " + n +
" bytes of the file one read() at a time");
for (int i=0; i < n; i++) {
System.out.print((char) f.read());
}
System.out.println("\nStill Available: " + f.available());
System.out.println("Reading the next " + n + " with one read(b[])");
byte b[] = new byte[n];
if (f.read(b) != n) {
System.err.println("couldn't read " + n + " bytes.");
}
System.out.println(new String(b, 0, n));
System.out.println("\nStill Available:"+ (size = f.available()));
System.out.println("Skipping half of remaining bytes with skip()");
f.skip(size/2);
System.out.println("Still Available: " + f.available());
System.out.println("Reading"+ n/2 + " into the end of array");
if (f.read(b, n/2, n/2) != n/2) {
System.err.println("couldn't read " + n/2 + " bytes.");
}
System.out.println(new String(b, 0, b.length));
System.out.println("\nStill Available: " + f.available());
f.close();
}
}
```

❑ FileOutputStream

- FileOutputStream is used to write bytes to a file.
- It creates an OutputStream connected to a file.
- Constructors

  FileOutputStream(String filePath)
  FileOutputStream(File fileObj)
  FileOutputStream(String filePath, boolean append)
  FileOutputStream(File fileObj, boolean append)

```
// Demonstrate FileOutputStream.
import java.io.*;
class FileOutputStreamDemo {
public static void main(String args[]) throws Exception
{
String source = "Now is the time for all good men\n"
+ " to come to the aid of their country\n"
+ " and pay their due taxes.";
byte buf[] = source.getBytes();
OutputStream f0 = new FileOutputStream("file1.txt");
for (int i=0; i < buf.length; i += 2) {
f0.write(buf[i]);
}
f0.close();
OutputStream f1 = new FileOutputStream("file2.txt");
f1.write(buf);
f1.close();
OutputStream f2 = new FileOutputStream("file3.txt");
f2.write(buf,buf.length-buf.length/4,buf.length/4);
f2.close();
}
}
```

❑ Reader

- Reader is an abstract class used for character input in Java.
- It defines the basic methods for reading character streams.
- All methods in the Reader class throw IOException when an error occurs.

Method	Description
void close()	Closes the input stream; further read attempts cause an IOException.
void mark(int numChars)	Marks the current position in the stream.
boolean markSupported()	Checks whether mark() and reset() are supported
int read()	Reads the next character; returns **-1** at end of file.
int read(char buffer[])	Reads characters into a buffer array.
boolean ready()	Returns true if the next input operation will not wait.
void reset()	Resets the stream to the previously marked position.
long skip(long numChars)	Skips the specified number of characters in the input stream.

❑ Writer:-

- Writer is an abstract class used for character output in Java.
- It defines methods for writing character streams.
- All methods in the Writer class return void.
- If an error occurs, the methods throw an IOException.

Method	Description
void close()	Closes the output stream; further write attempts cause an IOException.
void flush()	Clears output buffers and finalizes the output.
void write(int ch)	Writes a single character to the output stream.
void write(char buffer[])	Writes a complete array of characters to the invoking output stream.
void write(String str)	Writes str to the invoking output stream.
void write(String str, int offset, int numChars)	Writes a subrange of numChars characters from the array str, beginning at the specified offset.

❑ FileReader

- FileReader is used to read character data from a file.
- It creates a Reader stream connected to a file.
- It is used for character-oriented file input operations.
- Constructors

FileReader(String filePath)
FileReader(File fileObj)

```
// Demonstrate FileReader.
import java.io.*;
class FileReaderDemo {
public static void main(String args[])
throws Exception {
FileReader fr = new
FileReader("FileReaderDemo.java");
BufferedReader br = new
BufferedReader(fr);
String s; while((s = br.readLine()) != null)
{
System.out.println(s);
}
fr.close();
}
}
```

❑ FileWriter

- FileWriter is used to write character data to a file.
- It creates a Writer stream connected to a file.
- It is used for character-oriented file output operations.
- **Constructors**

FileWriter(String filePath)
FileWriter(String filePath, boolean append)
FileWriter(File fileObj)
FileWriter(File fileObj, boolean append)

```
// Demonstrate FileWriter.
import java.io.*;
class FileWriterDemo {
public static void main(String args[])
throws Exception {
String source = "Now is the time for all
good men\n" + " to come to the aid of
their country\n" + " and pay their due
taxes.";
char buffer[] = new char[source.length()];
source.getChars(0, source.length(), buffer,
0);
FileWriter f0 = new FileWriter("file1.txt");
for (int i=0; i < buffer.length; i += 2) {
f0.write(buffer[i]);
}
f0.close();
FileWriter f1 = new
FileWriter("file2.txt");
f1.write(buffer);
f1.close();
FileWriter f2 = new
FileWriter("file3.txt");
f2.write(buffer,buffer.length-
buffer.length/4,buffer.length/4);
f2.close();
}
}
```

❑ Using Stream I/O

```
// A word counting utility.
import java.io.*;
class WordCount {
public static int words = 0;
public static int lines = 0;
public static int chars = 0;
public static void wc(InputStreamReader isr)
throws IOException {
int c = 0;
boolean lastWhite = true;
String whiteSpace = " \t\n\r";
while ((c = isr.read()) != -1) {
// Count characters
chars++;
if (c == '\n') {
lines++; }
int index = whiteSpace.indexOf(c);
if(index == -1) {
if(lastWhite == true) {
++words; }
lastWhite = false; }
else {
lastWhite = true;
} }
if(chars != 0) {
++lines;
} }
public static void main(String args[]) { FileReader fr;
try {
if (args.length == 0) { // We're working with stdin
wc(new InputStreamReader(System.in));
}
else { // We're working with a list of files
for (int i = 0; i < args.length; i++) {
fr = new FileReader(args[i]);
wc(fr);
} } }
catch (IOException e) {
return;
}
System.out.println(lines + " " + words + " " + chars);
} }
```

❑ The Java Thread Model:-

- Java uses threads to perform multiple tasks at the same time.
- Multithreading makes Java programs more efficient and reduces CPU idle time.
- In a single-threaded system, only one task runs at a time using an event loop & If that single thread stops or waits for a resource, the entire program stops.
- In multithreading, if one thread waits other threads continue running.
- A thread can be in different states:

➢Running – the thread is executing.

➢Ready – waiting for CPU time.

➢Blocked – waiting for a resource.

➢Suspended – temporarily paused.

➢Terminated – thread has finished execution.

❑ Thread Priorities:-

- In Java, every thread has a priority.
- Thread priority determines which thread gets CPU time first.
- A higher-priority thread is given preference over lower-priority threads.
- A context switch occurs when the CPU switches from one thread to another.
- Context switching happens in two main ways:
  - A thread voluntarily gives up CPU by yielding, sleeping, or waiting for I/O.
  - A higher-priority thread preempts a lower-priority thread (preemptive multitasking).
- If two threads have the same priority, the CPU may divide time between them, depending on the operating system.

❑ Thread Class and Runnable Interface:

- Java's multithreading system is based on the Thread class and the Runnable interface.
- The Thread class represents a thread of execution.
- A running thread is controlled through a Thread object (instance).
- A new thread can be created in two ways:
  - By extending the Thread class
  - By implementing the Runnable interface
- Important methods of the Thread class:

Method	Meaning
getName()	Returns the name of the thread
getPriority()	Returns the priority of the thread
isAlive()	Checks if the thread is still running
join()	Waits for a thread to finish execution
run()	Entry point of the thread
sleep()	Pauses the thread for a specific time
start()	Starts the thread and calls the run() method

❑ The Main Thread

- When a Java program starts, a main thread begins running automatically.
- It is called the main thread because program execution starts from it.
- The main thread is important for two reasons:
  - It is the thread from which other “child” threads will be spawned.
  - Often it must be the last thread to finish execution because it performs various shutdown actions.
- The main thread is created automatically but can be controlled using a Thread object.
- A reference to the main thread can be obtained using the method:

  static Thread currentThread()

- This method is a public static member of the Thread class.

❑ Creating a Thread:-

- Java provides two ways to create a thread:
  - Implement the Runnable interface
  - Extend the Thread class

❑Implementing Runnable

- A class that implements Runnable must define the method:

  public void run()

- The **run()** method contains the code that will execute in the new thread.
- The thread starts execution from the run() method.
- The thread ends when the run() method finishes.
- After implementing Runnable, create a Thread object using the constructor:

  Thread(Runnable threadOb, String threadName)

- **threadOb** :- object that implements Runnable.
- **threadName** :- name of the new thread.
- The thread starts executing when the start() method is called.
- start() automatically calls the run() method and begins the thread execution.

```
// Create a second thread.
class NewThread implements Runnable {
Thread t;
NewThread() {
// Create a new, second thread
t = new Thread(this, "Demo Thread");
System.out.println("Child thread: " + t);
t.start(); // Start the thread
}
// This is the entry point for the second thread.
public void run() {
try {
for(int i = 5; i > 0; i--) {
System.out.println("Child Thread: " + i);
Thread.sleep(500);
}
} catch (InterruptedException e) {
System.out.println("Child interrupted.");
}
System.out.println("Exiting child thread.");
}
}
class ThreadDemo {
public static void main(String args[]) {
new NewThread(); // create a new thread
try {
for(int i = 5; i > 0; i--) {
System.out.println("Main Thread: " + i);
Thread.sleep(1000);
}
} catch (InterruptedException e) {
System.out.println("Main thread interrupted.");
}
System.out.println("Main thread exiting.");
}
}
```

❑ Extending Thread:-

- A thread can also be created by extending the Thread class.
- The new class must override the run() method.
- The run() method contains the code that the thread will execute.
- A thread object is created from the class.

```
// Create a second thread by extending Thread
class NewThread extends Thread {
NewThread() {
super("Demo Thread");
System.out.println("Child thread: " + this);
start(); // Start the thread
}
public void run() {
try {
for(int i = 5; i > 0; i--) {
System.out.println("Child Thread: " + i);
Thread.sleep(500); }
} catch (InterruptedException e) {
System.out.println("Child interrupted."); }
System.out.println("Exiting child thread.");
} }
class ExtendThread {
public static void main(String args[]) {
new NewThread(); // create a new thread
try {
for(int i = 5; i > 0; i--) {
System.out.println("Main Thread: " + i);
Thread.sleep(1000);
}
} catch (InterruptedException e) {
System.out.println("Main thread interrupted.");
}
System.out.println("Main thread exiting.");
} }
```

❑ Creating Multiple Thread:-

```
// Create multiple threads.
class NewThread implements Runnable {
String name; // name of thread
Thread t;
NewThread(String threadname) {
name = threadname;
t = new Thread(this, name);
System.out.println("New thread: " + t);
t.start(); // Start the thread
}
// This is the entry point for thread.
public void run() {
try {
for(int i = 5; i > 0; i--) {
System.out.println(name + ": " + i);
Thread.sleep(1000);
}
} catch (InterruptedException e) {
System.out.println(name + "Interrupted");
}
System.out.println(name + " exiting.");
} }
class MultiThreadDemo {
public static void main(String args[]) {
new NewThread("One"); // start threads
new NewThread("Two");
new NewThread("Three");
try {
// wait for other threads to end
Thread.sleep(10000);
} catch (InterruptedException e) {
System.out.println("Main thread Interrupted");
}
System.out.println("Main thread exiting.");
}
}
```

❑ Synchronization

- Synchronization is used when multiple threads share the same resource.
- It ensures that only one thread accesses the resource at a time.
- Synchronization prevents data inconsistency and conflicts.
- Java uses a concept called a **monitor** (mutex lock) for synchronization.
- Only one thread can hold the monitor lock at a time.
- When a thread acquires the lock, it enters the monitor & other threads trying to access it must wait until the lock is released.
- Java provides built-in language support for synchronization.
- Synchronization is implemented using the **synchronized** keyword.

❑ Using Synchronized Methods

```
class Callme {
void call(String msg) {
System.out.print("[" + msg);
try {
Thread.sleep(1000);
} catch(InterruptedException e) {
System.out.println("Interrupted");
}
System.out.println("]");
}
}
class Caller implements Runnable {
String msg;
Callme target;
Thread t;
public Caller(Callme targ, String s) {
target = targ;
msg = s;
t = new Thread(this);
t.start();
}
public void run() {
target.call(msg);
 }
}
class Synch {
public static void main(String args[]) {
Callme target = new Callme();
Caller ob1 = new Caller(target, "Hello");
Caller ob2 = new Caller(target, "Synchronized");
Caller ob3 = new Caller(target, "World");
// wait for threads to end
try {
ob1.t.join();
ob2.t.join();
ob3.t.join();
} catch(InterruptedException e) {
System.out.println("Interrupted");
}
}
}
```

❑ The synchronized statement

This is the general form of the synchronized statement:

synchronized(object) { // statements to be synchronized }

```
// This program uses a synchronized block.
class Callme {
void call(String msg) {
System.out.print("[" + msg);
try { Thread.sleep(1000);
} catch (InterruptedException e) {
System.out.println("Interrupted");
}
System.out.println("]");
} }
class Caller implements Runnable {
String msg;
Callme target;
Thread t;
public Caller(Callme targ, String s) {
target = targ;
msg = s;
t = new Thread(this);
t.start();
}
// synchronize calls to call()
public void run() {
synchronized(target) { // synchronized block
target.call(msg);
}
} }
class Synch1 {
public static void main(String args[]) {
Callme target = new Callme();
Caller ob1 = new Caller(target, "Hello");
Caller ob2 = new Caller(target, "Synchronized");
Caller ob3 = new Caller(target, "World");
// wait for threads to end
try {
ob1.t.join();
ob2.t.join();
ob3.t.join();
} catch(InterruptedException e) { System.out.println("Interrupted");
} } }
```

References

- Herbert Schildt, Java 2: The Complete Reference, McGraw-Hill , 5th Edition, 2018.
- Cay S. Horstmann, Gary Cornell, *Core Java Volume I - Fundamentals*, Pearson Education , 11th Edition, 2021
- Grady Booch, Robert A. Maksimchuk, Michael W. Engle, *Object-Oriented Analysis and Design with Applications*, Addison-Wesley , 3rd Edition, 2007
- E. Balagurusamy, *Programming with Java: A Primer*, McGraw Hill , 6th Edition, 2019
- Erich Gamma, Richard Helm, Ralph Johnson, John Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*, Addison-Wesley , 1st Edition, 1994
- PPT available for the respective books

THANK YOU